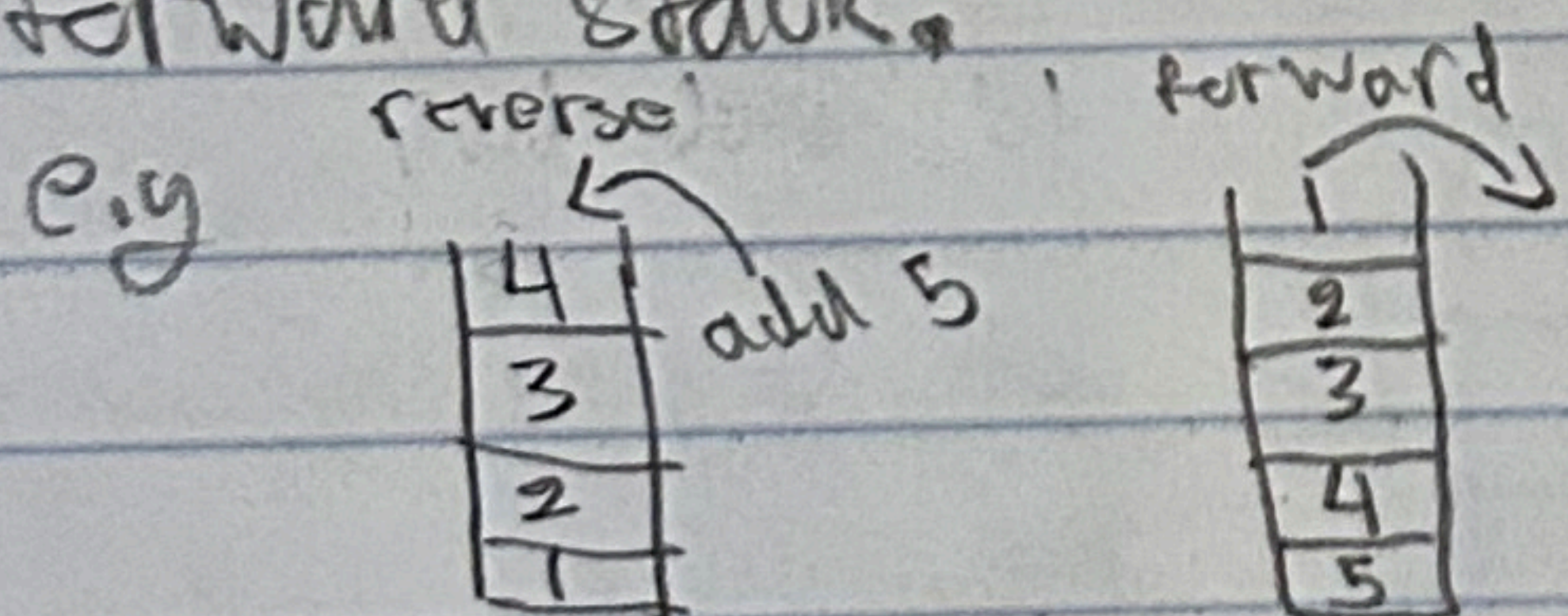


## Hw 2

1. From the algorithm, when there are 5 disks, there is 63 moves, this means the time complexity is  $O(2^n - 1)$  which reduces to  $O(2^n)$
2. The optimal value of  $k$  occurs around 16 in my tests. This occurs because when the  $k$  value is smaller, the algorithm is just going through merge sort. Merge sort is best for larger arrays but becomes much slower for smaller ones. This is a similar reason for when the  $k$  value is too high. When it is too high, the algorithm does more insertion sort. This method is much less effective with more values, slowing down the amount of time taken.
3. The maximum number of comparisons for array  $n$  is  $\frac{n(n-1)}{2}$ . Compared to bubble sort, merge sort has the same worst case maximum number of swaps.
4. To implement a queue using two stacks, you would need to have one stack be used for reverse order and another for forward order. When you need to add a value (enqueue), you put the value in the reverse order stack. If you need to remove a value, then you would use the forward stack.



reverse gives the ability to add while forward gives the ability to subtract

### 5. Pseudocode

list1 list2

compare head node 1 to head node 2

If  $hn1 > hn2$

new head node =  $hn2$

move to next node of list2

move to next node of new list

else

new list node =  $hn1$

move to next node of new list and list1

→ If next node of list 1 = null

while next node of 2  $\neq$  null

Set rest of list 2 to new list nodes

else if next node of list 2 = null

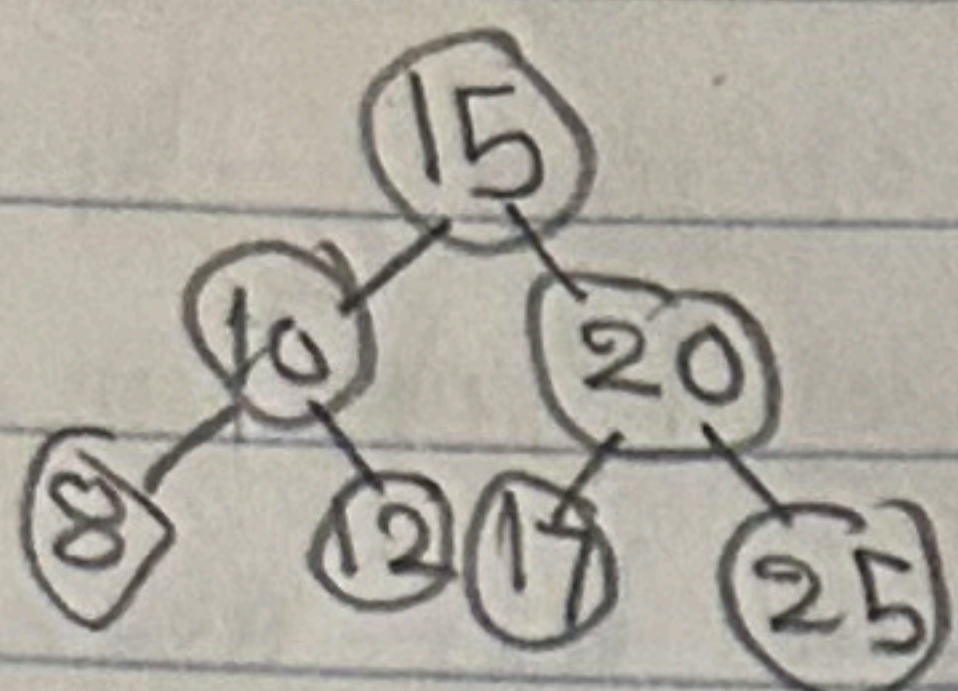
while next node of 1  $\neq$  null

Set rest of list 1 to new list nodes  
return new list

return new list to head node

6) For this algorithm, one solution is to split the unsorted matrix into arrays of the diagonals and temporarily store them. In this temporary storage, sort them from smallest by comparing each element and replacing them into the matrix, repeat this for  $M+N-1$  times.

7. a)



b) This binary tree is both complete and full. It is full because all nodes have either 2 leaf nodes or zero leaf nodes, it is complete since it has the max number of nodes on both the left and right sides and the leftmost side filled up first.

c) In-order traversal: 8, 10, 12, 15, 17, 20, 25

When outputting the tree with in-order traversal, it orders the array in ascending order.